

00_orquestador_pipeline

May 9, 2026

1 00. Orquestador del Pipeline

1.1 Goal

Execute the pipeline notebooks in order from a single notebook.

1.2 Notes

- This notebook is the execution entrypoint.
- 00_pipeline_mapa.ipynb remains only as visual documentation of the flow.
- Each target notebook is executed in its own folder so relative paths keep working.
- By default, execution stops on the first error.

```
[1]: from pathlib import Path
import subprocess
import sys
from datetime import datetime

ROOT_DIR = Path.cwd().resolve()

PIPELINE_NOTEBOOKS = [
    ROOT_DIR / '01_data_ingestion_enrichment' / '01_profiling_fuentes_validacion_join.ipynb',
    ROOT_DIR / '02_data_cleaning' / '01_staging_empresas_normalizadas.ipynb',
    ROOT_DIR / '02_data_cleaning' / '02_matching_exacto_scvs.ipynb',
    ROOT_DIR / '02_data_cleaning' / '03_catalogos_y_torneo_fuzzy.ipynb',
    ROOT_DIR / '02_data_cleaning' / '04_auditoria_llm_y_match_final.ipynb',
    ROOT_DIR / '02_data_cleaning' / '05_utilidad_consulta_ruc.ipynb',
    ROOT_DIR / '03_feature_engineering' / '01_base_analitica_empresas.ipynb',
    ROOT_DIR / '03_feature_engineering' / '02_features_fuentes.ipynb',
    ROOT_DIR / '03_feature_engineering' / '03_matriz_final_clustering.ipynb',
]

RUN_ONLY_EXISTING = True
STOP_ON_ERROR = True
TIMEOUT_SECONDS = 3600

print('Root:', ROOT_DIR)
print('Notebooks in pipeline:', len(PIPELINE_NOTEBOOKS))
```

```
for i, nb in enumerate(PIPELINE_NOTEBOOKS, 1):
    print(f'{i:02d}.', nb.relative_to(ROOT_DIR))
```

Root: E:\TESIS MAESTRIA\Desarrollo_clustering_maestria

Notebooks in pipeline: 9

```
01. 01_data_ingestion_enrichment\01_profiling_fuentes_validacion_join.ipynb
02. 02_data_cleaning\01_staging_empresas_normalizadas.ipynb
03. 02_data_cleaning\02_matching_exacto_scvs.ipynb
04. 02_data_cleaning\03_catalogos_y_torneo_fuzzy.ipynb
05. 02_data_cleaning\04_auditoria_llm_y_match_final.ipynb
06. 02_data_cleaning\05_utilidad_consulta_ruc.ipynb
07. 03_feature_engineering\01_base_analitica_empresas.ipynb
08. 03_feature_engineering\02_features_fuentes.ipynb
09. 03_feature_engineering\03_matriz_final_clustering.ipynb
```

1.3 Optional Tips

- If you want to skip a notebook, remove it from PIPELINE_NOTEBOOKS.
- If you only want a partial run, keep the slice you need.
- If one notebook contains a paid API call, leave it in the list only when you really want that step.

```
[ ]: def execute_notebook(nb_path: Path, timeout_seconds: int = TIMEOUT_SECONDS) ->
↳ subprocess.CompletedProcess:
    if not nb_path.exists():
        raise FileNotFoundError(f'Notebook not found: {nb_path}')

    cmd = [
        'jupyter', 'nbconvert',
        '--to', 'notebook',
        '--execute',
        '--inplace',
        f'--ExecutePreprocessor.timeout={timeout_seconds}',
        nb_path.name,
    ]

    print('\n' + '=' * 90)
    print('Running:', nb_path.relative_to(ROOT_DIR))
    print('Started:', datetime.now().strftime('%Y-%m-%d %H:%M:%S'))

    result = subprocess.run(
        cmd,
        cwd=str(nb_path.parent),
        capture_output=True,
        text=True,
        encoding='utf-8',
        errors='replace',
    )
```

```

print('Return code:', result.returncode)
if result.stdout.strip():
    print('\nSTDOUT:\n')
    print(result.stdout[-4000:])
if result.stderr.strip():
    print('\nSTDERR:\n')
    print(result.stderr[-4000:])

return result

```

```

[ ]: results = []

for nb_path in PIPELINE_NOTEBOOKS:
    if RUN_ONLY_EXISTING and not nb_path.exists():
        print('Skipping missing notebook:', nb_path)
        continue

    result = execute_notebook(nb_path)
    results.append({
        'notebook': str(nb_path.relative_to(ROOT_DIR)),
        'returncode': result.returncode,
    })

    if result.returncode != 0 and STOP_ON_ERROR:
        raise RuntimeError(f'Pipeline stopped due to error in {nb_path.
↳relative_to(ROOT_DIR)}')

print('\nPipeline summary:')
for row in results:
    print(row)

```